



Compiladores

Material 3

Tokenizer

“Código Lexer.”

Prof. Roger Rocha Ferreira

LEXER / TOKENIZER

```
TOKENS = [  
    ("ESPACO"           , r"\s+" ),  
    ("SE"                , r"\bse\b" ),  
    ("SENAO"            , r"\bsenao\b" ),  
    ("ENQUANTO"         , r"\benquanto\b" ),  
    ("ESCREVA"          , r"\bescreva\b" ),  
    ("RETORNO"          , r"\bretorna\b" ),  
    ("IGUALDADE"        , r"==" ),  
    ("IGUAL"            , r"=" ),  
    ("SOMA"             , r"\+" ),  
    ("SUB"              , r"-" ),  
    ("ABRE_CHAVE"       , r"\{" ),  
    ("FECHA_CHAVE"      , r"\}" ),  
    ("ABRE_PAR"         , r"\(" ),  
    ("FECHA_PAR"        , r"\)" ),  
    ("NUMERO"           , r"\d+" ),  
    ("TEXTO"            , r"\".*?\\"" ),  
    ("NOME"             , r"[a-zA-Z_][a-zA-Z0-9_]*" ),  
]
```

Tradução

Léxica

Sintática

Opcional

Semântica

Codific.

TRADUTOR DE STRING PARA REGULAR EXPRESSION

`re.compile(<string>)`

Analisa e traduz a string do padrão.

```
def compilar_tokens():  
    TOKENS_COMP = []  
    for tipo, padrao in TOKENS:  
        regex_comp = re.compile(padrao)  
        TOKENS_COMP.append((tipo, regex_comp))  
    return TOKENS_COMP
```

A partir do texto da expressão regular (e.g.: `r'[0-9]+'`):

1. Lê-se de caractere por caractere;
2. Valida a sintaxe; e
3. Converte em **autômato**.

Autômato é um Máquina de Estados Finitos (MEF) formada por um grafo com Estado Inicial e Transições que por fim gera um Estado de Aceitação.

LEXER

```
tokens = []
pos = 0
while pos < len(codigo):
    match = None
    for tipo, padrao in TOKENS_COMP:
        match = padrao.match(codigo, pos)
        if match:
            if tipo != "ESPACO":
                tokens.append(tipo)
            pos = match.end(0)
            break
    if not match:
        raise SyntaxError(f"Caractere inesperado: {codigo[pos]}")
```



Exercício

A. Implemente os fragmentos apresentados anteriormente em um tradutor incompleto para a análise léxica de um código em arquivo .mines.

B. Acrescente ao código os tokens para reconhecimento de:

1. Comentário de Linha (//);
2. Multiplicação (*); e
3. Divisão (/)
4. Leia;
5. Maior, Menor e negação.

C. Ao executar o programa, por intermédio do argumento “-coment”, ele deve considerar os Comentário de Linha de código.